

NeuroGrade AI

Project Architecture & Technical Implementation Guide

1. Executive Summary

NeuroGrade AI is a state-of-the-art academic prediction and analytics application designed to identify students at risk of academic failure. By analyzing key indicators such as study habits, wellness indices, and academic attendance, the platform utilizes machine learning algorithms to compute pass probabilities and provides customized behavioral strategies for both new and active students.

2. Technical Stack & Architecture

The project follows a decoupling strategy featuring a high-performance Flask API backend and a responsive glassmorphic React frontend:

- **Frontend (Vite + React):** Styled using modular Vanilla CSS adhering to a deep-purple glassmorphic aesthetic. Visualization is rendered dynamically using Recharts components (PieChart, BarChart, LineChart, Wellness Gauge). Icons are served by Lucide-React.
- **Backend (Flask + Python):** Serves a secure, modular REST API. Leverages Flask-SQLAlchemy for object relational mapping and Flask-JWT-Extended for authentication session security.
- **Database (SQLite / MySQL):** Employs SQLite locally for developer convenience with clean structures defined in SQLAlchemy to enable instant production migration to MySQL databases.
- **Machine Learning:** Executes binary classification via a pre-trained *Random Forest Classifier* built on 19 user-specific indicators, returning a confidence rating and pass/fail prediction.

3. System Workflows & User Lifecycles

The application is uniquely tailored to offer a seamless, personalized experience depending on user registration state:

A. Newly Registered / Guest Users

To protect data integrity, unauthenticated users cannot access predictions. When a guest tries to navigate to the predictor `/predict`, they are instantly intercepted by React route guards and redirected to `/register`. On registration, the backend analyzes their major and generates immediate, personalized learning tools and book list recommendations. Upon login, the Dashboard exhibits elegant, interactive empty states with custom

prompts directing them to perform their first evaluation.

B. Active / Registered Users

Once a user submits the **5-Step Prediction Form Wizard**, the input features are submitted alongside their JWT token. The model predicts the success rate, saves the student record linked to their user account, and outputs dynamic recommendations. The **Overview** (`/home``), **Dashboard** (`/dashboard``), and **Profile** (`/profile``) screens compile this history to display real average metrics, historical trendlines, motivation metrics, and achievement badges.

4. Key Data Models & Variables

The Random Forest model utilizes key features organized into five thematic categories to construct accurate predictions:

Category	Core Features Evaluated	Significance
Academic Info	Study Hours, Attendance %, Previous Grades, Assignment Completion	Measures direct scholastic dedication
Lifestyle	Sleep Patterns, Screen Time, Extra-curricular hours	Tracks baseline rest and energy allocation
Environment	Internet Access, Family Support, Study Space Quality	Measures exterior stability
Mindset	Stress Levels, Motivation, Class Participation	Correlates wellness with test success

5. Security & Protection

Security is enforced at both application layers:

- Backend Endpoint Shielding:** The prediction `/api/predict`` and statistics `/api/dashboard/stats`` endpoints are secured using `@jwt_required()`. Any requests lacking a valid JWT authorization token header are immediately blocked with a 401 response.
- Client-Side Interceptors:** The Axios HTTP client is equipped with interceptors that automatically catch 401 exceptions, purge expired local storage tokens, and redirect the browser safely to the login screen.
- Registration Guards:** Prompts users to register before making a prediction, protecting the machine learning pipeline from spam or anonymous abuse.

6. Future Enhancements & Developer Contacts

A preview of upcoming systems built into the landing page features list includes an **AI Chatbot Companion** for instant student mentorship, integrated study timers, and group collaboration workspaces.

For support or project details, you can contact our lead developers on Telegram:

- **Yoni** (Lead UI/UX Designer) — https://t.me/Yoni_yoi
- **H3B** (Lead Backend Architect) — <https://t.me/H3B6M9>
- **Juccj** (Lead AI Architect) — <https://t.me/juccj>

7. 7. Deployment & CI/CD

Overview of containerization with Docker, GitHub Actions workflow, and automated testing pipelines.

8. 8. Performance Optimizations

Techniques for lazy loading components, memoization, and server-side caching of predictions.

9. 9. Internationalization (i18n)

Implementation of language packs using react-i18next for multilingual support.

10. 10. Accessibility (a11y)

WCAG compliance in UI components, ARIA attributes, and keyboard navigation.

11. 11. Mobile Responsiveness

Responsive breakpoints, CSS Grid/Flex, and progressive web app manifest.

12. 12. Logging & Monitoring

Integration with Sentry for error tracking, and Prometheus for performance metrics.

13. 13. Testing Strategy

Jest unit tests for React components, Pytest for Flask routes, and coverage reports.

14. 14. Documentation & API Spec

OpenAPI spec generation via Flask-RESTX, and Swagger UI for front■end integration.

15. 15. Future Roadmap

Planned AI chatbot, real-time collaboration hub, and gamified learning modules.